

The Schema Changed and Nobody Told the Agent: Silent Failure and Diff-Conditioned Recovery under Tool-Schema Drift

Mohammed Abraar
Vizuara Research
abraar@vizz.vizuara.ai

Abstract

Tool-calling agents are built against a tool schema. The schema then versions: fields get renamed, enums get tightened, optional arguments become required, types change, defaults change. We measure what happens when the schema changes and the agent is not told. We freeze SchemaDrift-120, a testbed of 30 tools and 120 programmatically checkable tasks, and apply five deterministic drift classes as minimal pairs, so every drifted schema differs from its original by exactly one change. Four model families act as agents (Gemini 3.6 Flash, Gemini 3.1 Pro, GPT-5.6-luna, Qwen2.5-7B) across 4,800 episodes. First, drift is severe: task success falls from 87 to 99 percent at baseline to 11 to 38 percent when the executor silently enforces the new schema (all four paired drops $p < 5 \times 10^{-5}$). Second, whether the failure is visible depends jointly on the drift class and the API’s tolerance policy: default changes never raise an error under any validator, and a lenient executor converts 67 to 100 percent of rename and type drift into calls that are accepted with wrong semantics. A hand audit of 24 such calls confirms all 24, including an \$80 payment executed as EUR 80 and a two day access grant executed as four months. Third, a machine-readable schema diff in context, 13.7 percent of the schema’s token size, recovers 96 to 98 percent of the lost success on both Gemini models and outperforms re-injecting the full new schema (82 to 83 percent), because the diff carries the old contract, including deleted defaults, which the new schema alone does not. Fourth, the same diff recovers nothing on Qwen2.5-7B (0.06) and nothing on GPT-5.6-luna (point estimate -0.06): transcripts show both models keep emitting old-schema calls with the diff in context. Fifth, exposure itself is a property of calling style: GPT-5.6-luna fills every optional parameter explicitly and is therefore mechanically immune to newly-required and default-change drift (96 percent survival versus 25 to 31 percent for the Gemini models), while remaining fully exposed to renames, enum changes, and type changes. Schema drift is a correctness hazard that validators and retry loops structurally miss, and the cheapest repair, serving a diff, only works for models that read it.

1 Introduction

An agent that calls tools is executing someone else’s interface. The interface is described by a schema: a function name, typed parameters, enums, required flags, and documented defaults. The agent’s prompt carries a copy of that schema. The service behind the tool enforces its own copy. These two copies are assumed to be the same one.

In deployment they drift apart. Services version their APIs on their own schedule. A parameter is renamed, an enum value is retired, an optional field becomes mandatory, an integer becomes a formatted string, a default flips. The Model Context Protocol (MCP) ecosystem, where agents load tool declarations from independently maintained servers, makes this an everyday event rather than a rare migration (Owotogbe et al., 2026). Recent benchmarks

have established the headline consequence: agents degrade when tool interfaces evolve (Liu et al., 2026; Tian et al., 2026; Li et al., 2026a). What those benchmarks do not tell us is the part a practitioner most needs: when a drifted call fails, does anyone find out, and what is the cheapest thing that fixes it?

This paper answers both questions with a controlled instrument. We freeze a testbed before applying any drift: 30 tools across ten domains and 120 tasks whose correct calls are checkable by a script, with two phrasings per task. We then apply five deterministic drift classes as minimal pairs. Each drifted schema differs from its original by exactly one change: a parameter rename, an enum tightening, an optional parameter promoted to required, an integer changed to a formatted string, or a changed default. Because the transforms are deterministic and single-edit, every failure is attributable to a known cause. This is the difference between our instrument and the model-generated compound mutations of prior evolution benchmarks (Liu et al., 2026; Tian et al., 2026), which establish that degradation happens but cannot cleanly attribute it.

A local executor enforces the active schema and returns realistic error strings, or accepts the call. It runs under two tolerance profiles, because real APIs genuinely differ: a strict profile that rejects unknown parameters and type mismatches, and a lenient profile that silently drops unknown parameters and coerces types, which is common gateway behavior. Every drifted call lands in a three-way ledger: an error was surfaced, the call was accepted and correct, or the call was accepted with wrong semantics. The last cell is the one that should worry practitioners, and we hand-audit a sample of it.

On top of the drifted condition we compare three recovery levers, holding everything else fixed: (i) a structured, machine-readable diff of the schema change added to context, (ii) the full new schema replacing the stale one, and (iii) one retry with the raw executor error string. The diff is the cheap lever: it averages 28 tokens against 206 for a full schema declaration, 13.7 percent of the size, and in a large tool catalog it is the only artifact whose cost scales with the change rather than with the catalog.

Our contributions:

1. SchemaDrift-120, a frozen drift instrument: five deterministic minimal-pair drift classes over 30 tools and 120 checkable tasks, 48 task-drift pairs per class, with a dual-profile executor and a per-call outcome ledger (§3).
2. A drift-class by tolerance-policy map of silent failure (§4.1, §4.2): default changes surface no error under any validator; a lenient executor converts 67 to 100 percent of rename and type drift into accepted calls with wrong semantics; enum and newly-required drift reliably surface errors. All 24 audited wrong-semantics calls are confirmed wrong, several with money or access-control consequences.
3. A controlled three-way recovery comparison (§4.3): the diff recovers 0.96 to 0.98 of the lost success on Gemini models, beating full-schema re-injection (0.82 to 0.83) because the diff carries the old contract, and recovers approximately nothing on GPT-5.6-luna and Qwen2.5-7B, which ignore it. The raw-error retry recovers at most 0.39 anywhere and exactly nothing on the drift class that never errors.
4. Calling style as a mechanical exposure axis (§4.4): the share of optional parameters a model fills explicitly (0.45 to 1.00 across our families) predicts its survival on newly-required and default-change drift (0.25 to 0.96), a model-level property no prior drift work separates.

2 What was known and what is new

The premise that agents degrade under tool evolution is not ours. We list the closest prior work and the specific claim of ours that each does not cover.

What is new here: the deterministic minimal-pair drift taxonomy with per-class attribution, the error-surfaced versus accepted-wrong ledger conditional on a known induced drift class and executor policy, the controlled diff versus full-schema versus retry comparison, and the calling-style exposure axis.

Closest neighbor	What it established / what it does not cover
MCPEvol-Bench (Liu et al., 2026)	12 LLMs degrade 13 to 14 percent under LLM-simulated, backward-compatible MCP server evolution. No breaking minimal-pair transforms, no silent-failure accounting, no recovery lever that informs the agent of the change.
ToolBench-X (Tian et al., 2026)	Specification drift as one of five injected hazards; post-failure natural-language hints recover 60 to 80 percent. Drift instances are LLM-generated rewrites; hints are prose after task failure; its executor silently mutates payloads without measuring acceptance with wrong semantics.
ToolMisuseBench (Sigdel and Baral, 2026)	Deterministic fault injection including call-level drift, with rule-based schema repair. Mutates the emitted call, not the schema the agent believes; no LLM prompting levers; no wrong-semantics cell.
SilentProbe (Li et al., 2026b)	Silent (HTTP 200) failures of live production API tools; models detect 12 percent. Failures come from schema-prose gaps in static APIs, not from induced, attributable drift.
Diff-based code migration (Rosenfeld et al., 2025; Yang et al., 2026; Wu et al., 2025)	Diffs in context improve LLM code migration. Different task and failure surface; not a tool-calling agent at inference time, and no comparison against schema re-injection or error retry.
Silent-failure studies (Advani, 2026; Wu, 2026; Lin et al., 2026)	False success is common in agents generally. Not attributed to schema drift, and not conditioned on a known drift class.

Table 1: Novelty delineation. Full related work in §6.

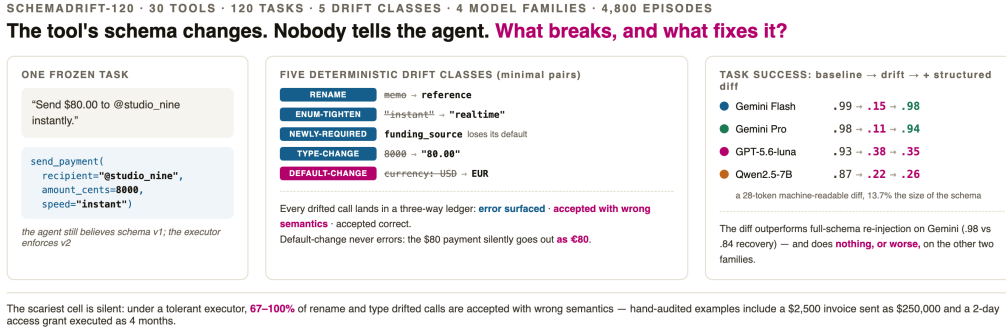


Figure 1: Study design and headline numbers. One frozen task, five deterministic drift classes applied to the tool schema as minimal pairs, a dual-profile executor with a three-way outcome ledger, and the measured effect of a 28-token schema diff on four model families.

Significance. Who is affected: anyone shipping agents against MCP servers or OpenAPI backends that version independently of the agent’s prompt. What changes: if a serving stack adds one machine-readable diff artifact to version bumps, the drift problem becomes nearly free to fix for model families that apply diffs, at about one seventh the token cost of re-serving schemas. And because default-change drift surfaces no error anywhere, monitoring built on error rates will report a healthy system while payments go out in the wrong currency; the accepted-wrong cell needs its own monitoring.

3 Method

Testbed. SchemaDrift-120 contains 30 tools, three in each of ten domains (calendar, payments, filesystem, CRM, messaging, web search, e-commerce, travel, HR, analytics). Each tool is a JSON-schema function declaration with 3 to 7 parameters and carries, by construction, at least one enum parameter, one optional parameter with a documented default, one integer parameter, and one renameable optional parameter, so every drift class applies to every tool. There are 120 tasks, four per tool: 90 single-step and 30 two-step (at most three agent turns, with tool results fed back). Each task is a natural-language request

plus frozen ground-truth calls. Correctness is checked by a script: canonicalized argument matching (sorted keys, numeric equivalence, ISO-8601 normalization, case-insensitive free strings) with active-schema defaults filled before comparison. Each task has two phrasings; phrasing one is primary and phrasing two feeds a robustness battery. The testbed was frozen, with a recorded hash, before any drifted episode ran. One amendment happened during a pre-drift baseline calibration pass and is documented in Appendix A.

Drift classes. Five deterministic transforms, each applied alone (Table 2). Assignment is balanced: every task carries two assigned classes that target parameters its ground truth actually uses, giving 240 task-drift pairs, 48 per class; a primary subset of 120 pairs (24 per class) is what every family runs, and the cheaper families run all 240.

Class	Transform	Example	Ground truth under v2
rename	optional param renamed	memo $\rightarrow$ reference	key renamed
enum-tighten	used enum value replaced	instant $\rightarrow$ realtime	new value
newly-required	optional param loses default	funding_source required	old default, explicit
type-change	int $\rightarrow$ formatted string	8000 $\rightarrow$ "80.00"	converted value
default-change	documented default changes	currency USD $\rightarrow$ EUR	old default, explicit

Table 2: The five drift classes. Each drifted schema differs from v1 by exactly this one change.

Executor and ledger. A local mock executor validates each call against the active schema and returns either a realistic error string (modeled on pydantic and Stripe error formats; verbatim in Appendix B) or acceptance, after which omitted optional parameters take the active schema’s defaults. Two tolerance profiles are both measured, because which drifts can surface an error is partly the API’s choice, not the model’s: strict rejects unknown parameters, missing required parameters, enum violations, and type mismatches; lenient silently drops unknown parameters and coerces representable types, and still rejects missing required parameters and enum violations. Every drifted call is classified as error-surfaced, accepted-correct, or accepted-wrong (accepted with no error while its executed semantics, after default filling, differ from the user’s intent).

Conditions. Per task-drift pair: (A) the agent sees v1 and the executor enforces v1, the baseline; (B) the agent sees v1 while the executor enforces v2, under both profiles, the silent-drift core; (C) as B strict, plus a structured JSON diff of the change in the system prompt; (D) the agent sees the full v2 schema; (E) as B strict, with the raw error string returned and one retry. The diff format is a per-field change record with old and new values; verbatim in Appendix B.

Models and protocol. Gemini 3.6 Flash and Gemini 3.1 Pro through the native API (temperature 0, thinking minimal and low respectively), GPT-5.6-luna through an OpenAI-compatible gateway (temperature 0), and Qwen2.5-7B-Instruct on vLLM with the hermes tool template (greedy) on a single A10G. One episode per record, randomized order, resume-safe. Flash and Qwen run the full grid (240 pairs, both profiles, plus batteries); Pro and GPT-5.6-luna run the primary 120 pairs. Total: 4,800 episodes. The full study cost under 8 US dollars of metered API spend plus 1.7 GPU hours.

Analysis. The unit is the within-pair paired difference: bootstrap 95 percent confidence intervals (10,000 resamples), two-sided sign-flip permutation tests (20,000 permutations), and paired d_z . Recovery fraction for lever X is $(\text{succ}(X) - \text{succ}(B)) / (\text{succ}(A) - \text{succ}(B))$ with bootstrap intervals over pairs. Drift-class by condition interactions are tested directly by permuting class labels over pairs. Benjamini-Hochberg correction runs over all 112 reported tests; 74 survive at the effective threshold $p \leq 0.0316$, and only survivors are bolded in tables. Every number in this paper is produced by one scripted analysis reading the raw episode logs.

Model	Task success					
	A base	B strict	B lenient	C +diff	D +v2 schema	E +retry
Gemini 3.6 Flash ($n=240$)	.992	.154	.204	.975	.838	.442
Gemini 3.1 Pro ($n=120$)	.975	.108	.192	.942	.825	.342
GPT-5.6-luna ($n=120$)	.933	.383	.442	.350	.692	.367
Qwen2.5-7B ($n=240$)	.867	.221	.263	.263	.646	.471

Table 3: Task success by condition. n is task-drift pairs; A is per task. Bold in columns C, D, E: the paired difference against B strict survives Benjamini-Hochberg over all 112 tests. Per-class tables in Appendix C.

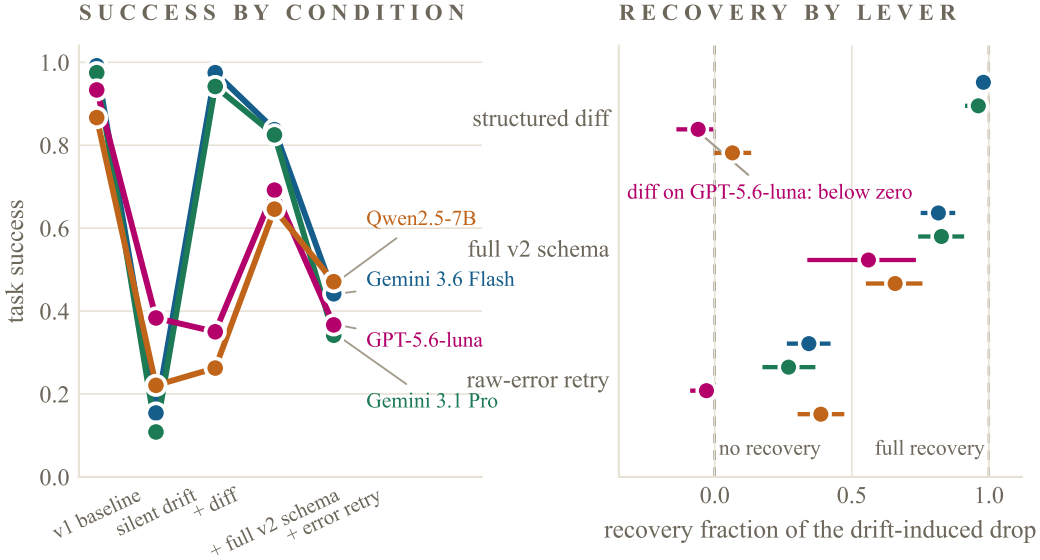


Figure 2: Left: success by condition for the four families. Right: recovery fraction of the drift-induced drop per lever, with bootstrap 95 percent intervals; right of the zero line means the lever recovered something. The diff (top row) splits the families: near-total recovery for both Gemini models, approximately zero for Qwen2.5-7B, and a negative point estimate for GPT-5.6-luna.

4 Results

4.1 Drift removes 55 to 87 points of task success

Table 3 and Figure 2 (left) give the study in one view. Baselines are high (0.87 to 0.99), so the tasks are within every family’s competence. Silent drift under the strict executor removes most of it: the paired drop is +0.84 [0.79, 0.88] for Flash, +0.87 [0.80, 0.93] for Pro, +0.55 [0.46, 0.63] for GPT-5.6-luna, and +0.65 [0.58, 0.70] for Qwen (all $p < 5 \times 10^{-5}$, d_z 1.10 to 2.54). The lenient executor changes the failure mode, not the level (success 0.19 to 0.44).

4.2 Whether drift is visible depends on the class and on the API’s tolerance

Figure 3 shows the outcome ledger for Flash (other families in Appendix C). Under the strict executor, four classes surface errors on essentially every failed call: rename, enum-tighten, newly-required, and type-change (85 to 100 percent error-surfaced). default-change is the exception: it violates nothing a validator can check, so it never errors anywhere; 38 percent of Flash’s default-change calls are accepted-wrong even under strict validation, and the remaining 62 percent survive only because the model happened to pass the cued value explicitly.

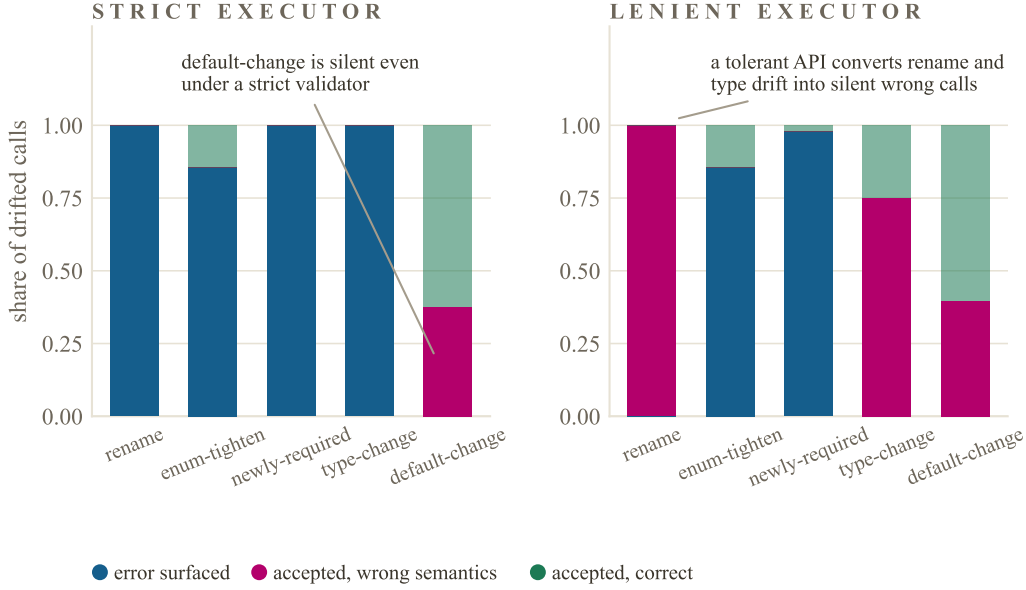


Figure 3: The outcome ledger for Gemini 3.6 Flash under silent drift, $n=48$ drifted calls per class. Left: strict executor. Right: lenient executor. Default-change never surfaces an error under either profile; the lenient profile converts rename and type drift into accepted calls with wrong semantics.

The lenient profile rewrites the map. Renamed parameters are silently dropped, so 100 percent of Flash’s rename-drift calls become accepted-wrong: the call succeeds and the renamed field’s content is simply gone, replaced by the new default. Type drift is coerced: 75 percent accepted with the wrong scale (an integer 8000 becomes the string "8000" where "80.00" was meant). The same tolerance policies that make APIs pleasant to integrate against convert two drift classes from loud to silent.

This has a direct consequence for the retry lever: a retry loop can only fix what errors. Per-class recovery for E on Flash is 0.98 on newly-required (the error names the missing parameter and the old default is documented in the stale schema) and 0.53 on enum-tighten (the error lists the allowed values), but 0.00 on rename, 0.04 on type-change (the error names the parameter but not the old-to-new mapping), and -0.06 on default-change, where there is no error to retry from.

4.3 The diff lever: near-total recovery, but only for families that read it

The right panel of Figure 2 is the paper’s central result. On the Gemini models the structured diff recovers 0.98 [0.95, 1.00] (Flash) and 0.96 [0.92, 0.99] (Pro) of the lost success, at a mean artifact size of 28 tokens against 206 for the full schema (13.7 percent). The diff beats full-schema re-injection (0.82 [0.76, 0.87] and 0.83 [0.75, 0.90]). The mechanism is visible in the per-class breakdown: on newly-required drift, D recovers only 0.40 on Flash while C recovers 1.00, because the new schema shows that the parameter is required but no longer contains the deleted default value; the diff carries the old contract, so the agent knows both that the field is now mandatory and what value preserves the old behavior. The same holds for default-change (C 0.89 versus D 0.83). A diff is not merely a compressed schema; it is strictly more informative about the past.

On the other two families the lever fails. Qwen2.5-7B: recovery 0.06 [0.01, 0.13], paired difference $+0.042$, $p = 0.064$, not significant after correction. GPT-5.6-luna: recovery point estimate -0.06 $[-0.13, -0.01]$, paired difference -0.033 , $p = 0.12$, also not significant after correction; what is established is the absence of recovery, with a negative point estimate. Transcripts show the failure is not subtle: under condition C both models keep emitting v1-style calls with the change log sitting in the system prompt, and on GPT-5.6-luna the

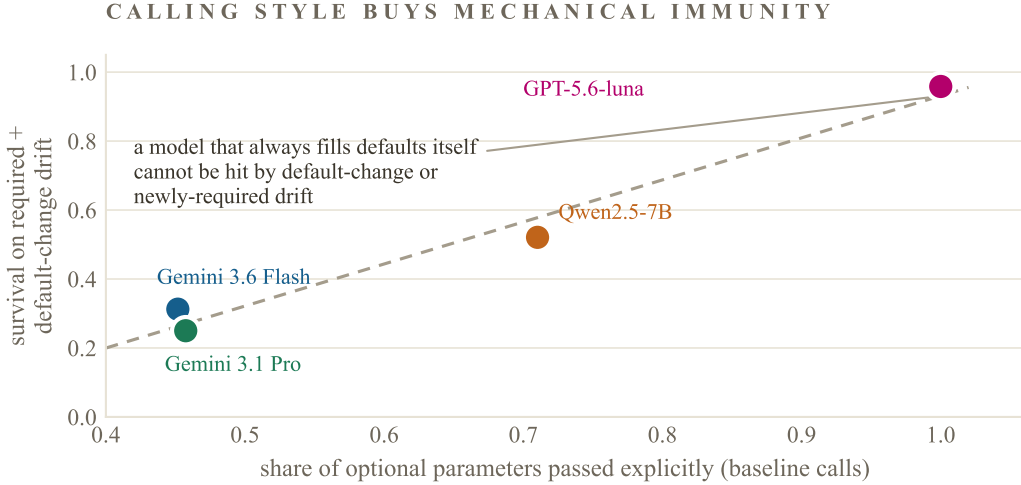


Figure 4: Share of optional parameters filled explicitly at baseline (x) against survival on newly-required plus default-change drift under silent drift (y), one point per family. Defensive filling is mechanical immunity to exactly these two classes; it does nothing for rename, enum, or type drift.

note occasionally perturbs calls on pairs it would otherwise have survived. Full-schema re-injection, by contrast, transfers everywhere (0.56 to 0.83, all surviving correction). The drift-class by condition interaction is significant for every lever on the Gemini models and for D and E on Qwen ($p < 5 \times 10^{-4}$ by label permutation), confirming that lever effectiveness genuinely differs by class rather than only by level.

4.4 Calling style is a mechanical exposure axis

Why does GPT-5.6-luna lose only 55 points where Pro loses 87? Not robustness in any adaptive sense. GPT-5.6-luna passes every optional parameter explicitly in its baseline calls (fill rate 1.00, against 0.45 for Flash, 0.46 for Pro, 0.71 for Qwen). A model that always writes currency="USD" itself cannot be hurt when the default changes, and cannot omit a parameter that just became required. Its survival on those two classes is 0.96, against 0.25 to 0.31 for the Gemini models; on the three classes that defensive filling cannot help (rename, enum, type), every family is equally destroyed (survival 0.00 to 0.05). Figure 4 plots the relationship across the four families. Exposure to two of the five drift classes is set by an argument-writing habit, before any reasoning about change happens.

4.5 The accepted-wrong cell, audited

Rates alone understate this cell, so we hand-verified a stratified sample of 24 accepted-wrong episodes across all four families and all five classes (audit file ships with the release). All 24 are confirmed: the call was accepted without any error and did something other than what the user asked. The concrete shapes: a payment of \$80.00 executed as EUR 80 after a currency default change; a \$375.50 invoice issued as \$37,550 and a \$2,500 invoice as \$250,000 after integer-cents to decimal-string type drift met lenient coercion; a two day access grant executed as a 2,880 hour grant; a Toronto job posting published as remote after a renamed location field was silently dropped; a customer never notified of a refund after a notification default flipped. Two of the 24 are low-stakes (a bookmark folder, a deal metadata field), and in two the wrong value was partly model-invented rather than purely mechanical; we label these in the audit file. Under the lenient executor these are exactly the calls a monitoring dashboard counts as successes.

4.6 Batteries: noise, phrasing, and correction

Thirty stratified pairs were re-run five times each under silent drift: 13.3 percent of Flash pairs flip success at least once across five identical calls at temperature 0; Qwen under greedy decoding flips zero. The headline effects (55 to 87 point drops, 60 to 82 point lever gaps) are far outside this noise floor. Re-running baseline and drift conditions on the second phrasing of every task moves success by at most 4.2 points (all $p \geq 0.27$), so the results are not a property of one phrasing. Benjamini-Hochberg over all 112 reported tests retains 74 at an effective threshold of $p \leq 0.0316$, including every effect named in the abstract; individual cells that fail correction are marked and should be read as unconfirmed.

5 Limitations, narrated

The executor is a mock. Its two tolerance profiles bracket real API behavior rather than sample it; the ledger’s per-class silent rates are therefore properties of drift class times policy, which is the claim, but the mix of policies in production is not measured here. The silent-failure composition we report is partly a design consequence: we chose type conversions where naive coercion yields a valid but wrongly scaled value, because that is the dangerous case; conversions that fail coercion would surface errors instead.

Our pre-registered directions did not all survive. We recorded, before any data, the direction that newly-required arguments would cause the largest silent failure rate. The opposite is true: newly-required drift is the most reliably surfaced class, and default-change is the silent one. We noted at registration time that the executor mechanics might force exactly this reversal, and they did; we report it as a reversal. The token-cost direction (diff under 10 percent of schema size) also missed: the measured ratio is 13.7 percent.

The diff result is a capability statement about four checkpoints in September 2026, not a law. The negative results on GPT-5.6-luna and Qwen do not survive multiple-comparison correction as harms; they are absences of benefit. We did not test whether prompting harder (for example, an instruction to apply the change log step by step) unlocks diff-following on those families; that is one experiment away and would sharpen the deployment advice.

The testbed is synthetic and English-only, with 30 tools and single-edit drift. Real drift arrives in bundles, with changelogs of mixed quality, and on tool catalogs an order of magnitude larger, where the token argument for diffs gets stronger but the retrieval problem (which diff applies to this call) appears. Two-step tasks cap at three turns. One phrasing battery and a five-repeat noise floor bound but do not eliminate stochastic and phrasing variance. The 24-call audit covers the cell’s variety, not its full population.

6 Related work

Tool evolution and drift. MCPEvol-Bench (Liu et al., 2026) and ProEvolve (Li et al., 2026a) establish degradation under evolving tool environments with LLM-driven mutations; ToolBench-X (Tian et al., 2026) injects specification drift among five hazards and shows prose hints help after failure; ToolMisuseBench (Sigdel and Baral, 2026) injects call-level drift deterministically with rule-based repair; ReliabilityBench (Gupta, 2026) and LayerRAG-Bench (Shams, 2026) include schema drift among stress faults; skill libraries decay as contracts change (Fan et al., 2026). None isolates single drift classes, none measures acceptance with wrong semantics, and none compares change-information levers.

Robustness and static benchmarks. The classic tool-use benchmarks evaluate static contracts (Qin et al., 2024; Li et al., 2023; Tang et al., 2023; Chen et al., 2023; Yao et al., 2024; Liu et al., 2024; Ruan et al., 2024; Huang et al., 2023; Yan et al., 2024). RoTBench (Ye et al., 2024) perturbs the schema the model is shown, adversarial noise on a static contract, whereas our drift is a believed-versus-enforced mismatch. StableToolBench (Guo et al., 2024) responds to real API instability by freezing it away, which is the strongest evidence the phenomenon matters. ToolRobustBench (Zheng et al., 2026) and ToolMaze (Zhu et al., 2026) perturb interfaces and runtime behavior at the tool level.

Silent failure. A 2026 wave documents false success in agents (Advani, 2026; Wu, 2026; Lin et al., 2026); SilentProbe (Li et al., 2026b) measures silent failure against live production APIs; an MCP audit finds error results rarely actionable (Mehan, 2026). We add the attribution: which induced drift class, under which tolerance policy, produces silence.

Recovery mechanisms. Error-feedback repair and reflection are well studied (Shinn et al., 2023; Gou et al., 2024; Su et al., 2025; Hamad et al., 2025; Ray and Goyal, 2026; Gumaan, 2026; Canedo and Chethan, 2026; Chen et al., 2026; Mazumder and jahan Lia, 2026); constrained decoding guarantees validity against the schema the client believes, which under drift validates against the wrong contract (Zhang et al., 2023). Diff-conditioning exists in code migration (Rosenfeld et al., 2025; Yang et al., 2026; Wu et al., 2025); we test the same lever for tool-calling agents at inference time, against re-injection and retry.

7 Conclusion

We versioned tool schemas out from under four agent families with five single-edit drift classes and an executor whose tolerance policy we controlled. Drift removes 55 to 87 points of task success. Whether anyone finds out depends on the class and the policy: default changes are silent everywhere, and a lenient gateway turns renames and type changes into confidently wrong calls, of which all 24 audited examples were genuinely wrong, some expensively. A 28-token machine-readable diff repairs nearly everything for the two models that apply it, and beats re-serving the full schema because it carries the deleted past. The same artifact does nothing for the two models that ignore it, and a model’s private habit of filling every optional argument decides, before any recovery lever runs, whether two of the five drift classes can touch it at all. The practitioner reading: version bumps should ship machine-readable diffs; error-rate monitoring does not cover default changes; and both the benefit of the diff and the exposure to drift are properties of the model family you deploy, so they belong in the evaluation you run before choosing it.

Reproducibility statement

The frozen testbed (with its hash), all drift transforms, the executor, every runner, the raw episode logs, the scripted analysis producing every number in this paper, the audit sample with verdicts, and the spend log accompany this submission and will be released publicly. Total marginal compute: under 8 US dollars of metered API spend and 1.7 GPU hours on one A10G.

LLM usage statement

Large language models are the subjects of this study (the four agent families under test) and served as engineering assistants for analysis code, figures, and manuscript editing. All experimental design choices, statistical decisions, interpretations, and claims are the authors’; every reported number was checked against the scripted analysis output before entering the text.

References

- Laksh Advani. From confident closing to silent failure: Characterizing false success in llm agents. arXiv preprint arXiv:2606.09863, 2026.
- Arquimedes Canedo and Grama Chethan. Self-reflective apis: Structure beats verbosity for ai agent recovery. arXiv preprint arXiv:2606.05037, 2026.
- Chaoran Chen, Vy Nguyen, Ziji Zhang, Abhinav Gullapalli, Ziyi Wang, Yuxuan Lu, Dakuo Wang, Jing Huang, Zhou Yu, and Jin Lai. Retry, switch, or abstain? learning strategy-aware tool-use policies via controlled error injection. arXiv preprint arXiv:2608.11977, 2026.
- Zehui Chen, Weihua Du, Wenwei Zhang, Kuikun Liu, Jiangning Liu, Miao Zheng, Jingming Zhuo, Songyang Zhang, Dahua Lin, Kai Chen, and Feng Zhao. T-eval: Evaluating

- the tool utilization capability of large language models step by step. arXiv preprint arXiv:2312.14033, 2023.
- Linfeng Fan, Yuan Tian, Ziwei Li, and Zhiwu Lu. Skill drift is contract violation: Proactive maintenance for llm agent skill libraries. arXiv preprint arXiv:2605.10990, 2026.
- Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. Critic: Large language models can self-correct with tool-interactive critiquing. In International Conference on Learning Representations, 2024. arXiv:2305.11738.
- Esmail Gumaan. Feedback that backfires: Why small language model agents repeat the call they just watched fail. arXiv preprint arXiv:2608.23651, 2026.
- Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models. arXiv preprint arXiv:2403.07714, 2024.
- Aayush Gupta. Reliabilitybench: Evaluating llm agent reliability under production-like stress conditions. arXiv preprint arXiv:2601.06112, 2026.
- Hassan Hamad, Yingru Xu, Liang Zhao, Wenbo Yan, and Narendra Gyanchandani. Tool-critic: Detecting and correcting tool-use errors in dialogue systems. arXiv preprint arXiv:2510.17052, 2025.
- Yue Huang, Jiawen Shi, Yuan Li, Chenrui Fan, Siyuan Wu, Qihui Zhang, Yixin Liu, Pan Zhou, Yao Wan, Neil Zhenqiang Gong, and Lichao Sun. Metatool benchmark for large language models: Deciding whether to use tools and which to use. arXiv preprint arXiv:2310.03128, 2023.
- Guangrui Li, Yaochen Xie, Yi Liu, Ziwei Dong, Xingyuan Pan, Tianqi Zheng, Jason Choi, Michael J. Morais, Binit Jha, Shaunak Mishra, Bingrou Zhou, Chen Luo, Monica Xiao Cheng, and Dawn Song. The world won’t stay still: Programmable evolution for agent benchmarks. arXiv preprint arXiv:2603.05910, 2026a.
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms. In Proceedings of the Conference on Empirical Methods in Natural Language Processing, 2023. arXiv:2304.08244.
- Zongrong Li, Shengkun Ye, Feiyu Guo, and Zuoyou Dang. Silentprobe: Measuring silent failure in production apis used as agent tools. arXiv preprint arXiv:2609.00035, 2026b.
- Xiaofeng Lin, Yingxu Wang, Tung Sum Thomas Kwok, Daniel Guo, Sahil Arun Nale, Charles Fleming, and Guang Cheng. Reflect: Intervention-supported error attribution for silent failures in llm agent traces. arXiv preprint arXiv:2606.09071, 2026.
- Huanxi Liu, Kun Hu, Jiaqi Liao, Qiang Wang, Pengfei Qian, YuanZhao Zhai, Dawei Feng, Bo Ding, and Huaimin Wang. Mcpevol-bench: Benchmarking llm agent performance across dynamic evolutions of mcp servers. arXiv preprint arXiv:2607.14642, 2026.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents. In International Conference on Learning Representations, 2024. arXiv:2308.03688.
- Aritra Mazumder and Nusrat jahan Lia. Agentcheck: A reproduce-intervene-mitigate workbench for llm agents over mcp. arXiv preprint arXiv:2607.11098, 2026.
- Rishabh Mehan. Can mcp clients decide what to do after failure? a result-only actionability audit. arXiv preprint arXiv:2609.00072, 2026.

- Joshua Owotogbe, Indika Kumara, Willem-Jan van den Heuvel, Damian Andrew Tamburri, Antonio Ken Iannillo, and Roberto Natella. A taxonomy of runtime faults in model context protocol servers. arXiv preprint arXiv:2606.05339, 2026.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis. In International Conference on Learning Representations, 2024. arXiv:2307.16789.
- Jaideep Ray and Ankit Goyal. Structured feedback improves repair in an llm agent loop. arXiv preprint arXiv:2607.14167, 2026.
- Katherine A. Rosenfeld, Cliff C. Kerr, and Jessica Lundin. What a diff makes: automating code migration with large language models. arXiv preprint arXiv:2511.00160, 2025.
- Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of lm agents with an lm-emulated sandbox. In International Conference on Learning Representations, 2024. arXiv:2309.15817.
- Musa Shams. Layerrag-bench: A cross-layer reliability benchmark for agentic retrieval-augmented generation. arXiv preprint arXiv:2607.27353, 2026.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In Advances in Neural Information Processing Systems, 2023. arXiv:2303.11366.
- Akshey Sigdel and Rista Baral. Toolmisusebench: An offline deterministic benchmark for tool misuse and recovery in agentic systems. arXiv preprint arXiv:2604.01508, 2026.
- Junhao Su, Yuanliang Wan, Junwei Yang, Hengyu Shi, Tianyang Han, Junfeng Luo, and Yurui Qiu. Failure makes the agent stronger: Enhancing accuracy through structured reflection for reliable tool interactions. arXiv preprint arXiv:2509.18847, 2025.
- Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, Boxi Cao, and Le Sun. Toolalpaca: Generalized tool learning for language models with 3000 simulated cases. arXiv preprint arXiv:2306.05301, 2023.
- Yang Tian, Zhengpeng Shi, Yu Zhou, and Bo Zhao. Beyond function calling: Benchmarking tool-using agents under tool-environment unreliability. arXiv preprint arXiv:2606.25819, 2026.
- Haoze Wu, Yunzhi Yao, Wenhao Yu, and Ningyu Zhang. Recode: Updating code api knowledge with reinforcement learning. arXiv preprint arXiv:2506.20495, 2025.
- Wei Wu. When errors become narratives: A longitudinal taxonomy of silent failures in a production llm agent runtime. arXiv preprint arXiv:2606.14589, 2026.
- Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Tianjun Zhang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Berkeley function-calling leaderboard. <https://gorilla.cs.berkeley.edu/leaderboard.html>, 2024.
- Shenghao Yang, Bo Lu, Yaochen Liu, Yu Kang, Qiongfang Zhang, Chetan Bansal, Saravan Rajmohan, and Minghua Ma. Deprepair: Llm-based source-code repair for dependency breaking changes. arXiv preprint arXiv:2607.17957, 2026.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. -bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv preprint arXiv:2406.12045, 2024.
- Junjie Ye, Yilong Wu, Songyang Gao, Caishuang Huang, Sixian Li, Guanyu Li, Xiaoran Fan, Qi Zhang, Tao Gui, and Xuanjing Huang. Rotbench: A multi-level benchmark for evaluating the robustness of large language models in tool learning. arXiv preprint arXiv:2401.08326, 2024.

Kexun Zhang, Hongqiao Chen, Lei Li, and William Wang. Don't fine-tune, decode: Syntax error-free tool use via constrained decoding. arXiv preprint arXiv:2310.07075, 2023.

YiShan Zheng, Yuan Wu, and Yi Chang. Toolrobustbench: Stage-wise perturbation evaluation and failure diagnosis for tool-calling agents. arXiv preprint arXiv:2608.23635, 2026.

Dongsheng Zhu, Xuchen Ma, Yucheng Shen, Xiang Li, Yukun Zhao, Shuaiqiang Wang, Lingyong Yan, and Dawei Yin. When tools fail: Benchmarking dynamic replanning and anomaly recovery in llm agents. arXiv preprint arXiv:2606.05806, 2026.

A Testbed provenance and the one amendment

SchemaDrift-120 was authored and frozen before any drifted episode ran. The frozen artifact is a single JSON file carrying the 30 v1 schemas, all 150 derived v2 schemas (30 tools times 5 classes), all structured diffs, the 120 tasks with both phrasings, the balanced class assignments, and the v1 and v2 ground truths; its SHA-256 hash is recorded in the project log and in the released repository (e8bbd71c63b1...d08475).

One amendment happened between the first freeze and the final one, during a baseline calibration pass that ran condition A only, before any drift condition had executed. Calibration on Gemini 3.6 Flash scored 114/120. The six failures were inspection-level artifacts, not capability failures: three case mismatches in free strings (fixed by making free-string comparison case-insensitive in the canonicalizer), one paraphrase invited by an unquoted reminder text, one prose phrase that leaked into an optional details field, and one ground-truth value inconsistent with its own phrasing. Three phrasings and one value were amended, the testbed re-frozen with a new hash, and all calibration and dry-run episodes discarded. No drifted arm had run at amendment time.

The 30 tools, three per domain: calendar (create_event, create_reminder, book_meeting_room); payments (send_payment, create_invoice, refund_payment); filesystem (copy_file, compress_folder, set_permissions); CRM (create_contact, log_interaction, update_deal); messaging (send_message, schedule_message, create_channel); web search (web_search, monitor_keyword, save_bookmark); e-commerce (place_order, update_listing, track_shipment); travel (book_flight, reserve_hotel, hail_ride); HR (request_time_off, post_job_opening, submit_expense); analytics (run_report, create_dashboard_widget, export_dataset).

B Verbatim prompts, diff format, and executor errors

System prompt (every condition).

You are a tool-calling assistant. Complete the user's request by calling the provided tools with correct arguments. Use parameter defaults by omitting a parameter when the user does not specify it. Amounts, durations, and sizes must be expressed in the units the schema asks for. Call one tool at a time; after all required calls are done, reply with the single word DONE.

Condition C addendum (appended to the system prompt; {diff} is the JSON below):

NOTE: the API behind tool '{tool}' was updated after the schema above was cached. Machine-readable change log for '{tool}' (apply it when calling):
{diff}

Diff format, one record per changed field; the five class instantiations:

```
[{"param": "memo", "change": "renamed", "old": "memo", "new": "reference"}]
[{"param": "speed", "change": "enum_changed", "old": "instant", "new": "realtime"}]
[{"param": "funding_source", "change": "now_required", "old": {"default": "default_balance"}, "new": "required, no default"}]
```

```
[{"param": "amount_cents", "change": "type_changed", "old": {"type": "integer", "desc": "Amount in cents"}, "new": {"type": "string", "desc": "Amount as decimal string in major units"}}]
[{"param": "currency", "change": "default_changed", "old": "USD", "new": "EUR"}]
```

Executor error strings (strict profile; lenient differs only as described in §3):

```
Received unknown parameter: 'location'.
Missing required parameter: 'calendar_id'.
Invalid value 'private' for parameter 'visibility'. Allowed values: public, restricted, confidential.
Invalid type for parameter 'duration_min': expected string, got integer.
```

Documented defaults are rendered into each parameter description as “(optional; default: X)” so all providers see identical schema information regardless of native schema-field support.

C Per-class results

Model	Cond	rename	enum	required	type	default
Gemini 3.6 Flash	B strict	.000	.146	.000	.000	.625
	C +diff	.979	1.000	1.000	.938	.958
	D +v2	1.000	.958	.396	.896	.938
	E +retry	.000	.583	.979	.042	.604
Gemini 3.1 Pro	B strict	.000	.042	.000	.000	.500
	C +diff	.917	1.000	.958	.958	.875
	D +v2	1.000	.958	.417	.958	.792
	E +retry	.000	.292	.917	.000	.500
GPT-5.6-luna	B strict	.000	.000	1.000	.000	.917
	C +diff	.000	.000	.958	.000	.792
	D +v2	.917	.833	.167	.750	.792
	E +retry	.000	.000	1.000	.000	.833
Qwen2.5-7B	B strict	.000	.062	.333	.000	.708
	C +diff	.000	.167	.333	.083	.729
	D +v2	.875	.812	.167	.646	.729
	E +retry	.000	.667	.771	.208	.708

Table 4: Task success by drift class ($n=48$ pairs per class for Flash and Qwen, 24 for Pro and GPT-5.6-luna). Two readings: GPT-5.6-luna’s columns for required and default are its calling-style immunity (§4.4), and D’s weakness on newly-required (.167 to .417 everywhere) is the deleted-default effect that the diff repairs (§4.3).

D The audited accepted-wrong sample

Twenty-four accepted-wrong episodes were sampled stratified over model and drift class and hand-verified against the task phrasing, the call made, and the executed semantics after default filling. All 24 were confirmed. The full audit file, with per-episode verdicts and notes, ships in the release (results/audit_accepted_wrong_sample.json). Selected entries:

E Batteries and correction detail

Noise floor. Thirty stratified task-drift pairs, condition B strict, five identical calls each. Gemini 3.6 Flash at temperature 0: 13.3 percent of pairs flip task success at least once across the five calls, and the same share flips its ledger outcome. Qwen2.5-7B under greedy decoding: zero flips on both. Headline paired effects in this paper range from 23 to 87

Model	Class	Strict executor			Lenient executor		
		err	wrong	ok	err	wrong	ok
Flash	rename	1.00	.00	.00	.00	1.00	.00
	enum-tighten	.85	.00	.15	.85	.00	.15
	newly-required	1.00	.00	.00	.98	.00	.02
	type-change	1.00	.00	.00	.00	.75	.25
	default-change	.00	.38	.62	.00	.40	.60
Pro	rename	1.00	.00	.00	.00	1.00	.00
	enum-tighten	.96	.00	.04	.96	.00	.04
	newly-required	1.00	.00	.00	1.00	.00	.00
	type-change	1.00	.00	.00	.00	.67	.33
	default-change	.00	.46	.54	.00	.38	.62
GPT-5.6-luna	rename	1.00	.00	.00	.00	1.00	.00
	enum-tighten	1.00	.00	.00	1.00	.00	.00
	newly-required	.00	.00	1.00	.00	.00	1.00
	type-change	1.00	.00	.00	.00	.67	.33
	default-change	.00	.00	1.00	.00	.04	.96
Qwen2.5-7B	rename	.98	.02	.00	.00	1.00	.00
	enum-tighten	.94	.00	.06	.94	.00	.06
	newly-required	.58	.06	.35	.58	.06	.35
	type-change	1.00	.00	.00	.00	.79	.21
	default-change	.00	.27	.73	.00	.27	.73

Table 5: Full outcome ledger under silent drift (condition B): share of drifted calls per class whose outcome was error surfaced (err), accepted with wrong semantics (wrong), or accepted correct (ok). Rows may not sum to 1.00 where the model made no call on some episodes; shares are over drifted calls made.

Model	Class	Executor	What executed
Gemini Pro	default-change	strict	\$80.00 payment executed with currency EUR
GPT-5.6-luna	type-change	lenient	\$2,500.00 invoice issued as \$250,000 (cents coerced to major units)
Gemini Flash	type-change	lenient	\$375.50 invoice issued as \$37,550
GPT-5.6-luna	type-change	lenient	2-day access grant executed as 2,880 hours
Gemini Flash	rename	lenient	job_location dropped; Toronto job posted with default "remote"
Gemini Flash	default-change	lenient	customer not notified of a refund they requested
Qwen2.5-7B	type-change	lenient	\$12.75 expense filed as \$1,275
Gemini Flash	rename	lenient	gift note dropped from an order; recipient gets no message
Qwen2.5-7B	required	strict	model invented currency="EUR" for a German IBAN; accepted

Table 6: Nine of the 24 audited episodes. Two of the 24 are low-stakes metadata errors; in two the wrong value was partly model-invented rather than purely mechanical (both labelled in the audit file).

points and sit far outside this floor; individual per-class cells within a few points of each other should not be over-read.

Phrasing battery. Conditions A and B strict re-run on phrasing 2 of all 120 tasks (Flash and Qwen). Paired shift phrasing 2 minus phrasing 1: Flash A +0.008 ($p=1.0$), Flash B strict -0.042 ($p=0.27$); Qwen +0.008 on both ($p=1.0$).

Interaction tests. Drift-class by condition interaction, permutation of class labels over pairs (2,000 permutations, between-class variance statistic): Gemini Flash and Pro, all three levers $p < 5 \times 10^{-4}$; Qwen D and E $p < 5 \times 10^{-4}$, C $p=0.37$; GPT-5.6-luna D $p < 5 \times 10^{-4}$,

C $p=0.13$, E $p=0.19$. Where a lever does nothing overall, its per-class profile is flat, which is what the non-significant interactions say.

Multiple comparisons. Benjamini-Hochberg at $q=0.05$ over all 112 reported sign-flip tests retains 74 at an effective threshold of $p \leq 0.0316$. Every effect named in the abstract survives. The GPT-5.6-luna diff decrement (-0.033 , $p=0.12$) and the Qwen diff increment ($+0.042$, $p=0.06$) do not survive and are reported as absences of recovery, not as harms or gains.

Token accounting. Mean artifact size over all 150 tool-class pairs: structured diff 28.1 tokens, full v2 declaration 205.9 tokens (chars/4 estimate), ratio 0.137. Measured mean prompt tokens per call by condition ship in the released analysis output.